Protocoles Réseaux

Prof. Abderrahim Benslimane
Université d'Avignon

→ Licence 3ème Année

→ 2025-2026

Plan

Programmation de protocoles réseaux

Objectif :

Le but de ce module est d'initier l'étudiant à la programmation inter-processus sous UNIX. L'utilisation de l'API SOCKET est à la base avec les protocoles de transport TCP/IP et UDP/IP.

Contenu :

•Introduction à la communication répartie inter-processus
•Modèles client/serveur (2-tier et 3-tier) : côté application
•Rappel TCP, UDP
•Fichiers d'inclusion et Fonctions de la bibliothèque SOCKET
•Accès simultanés
•Applications : projet

Prérequis :

Initiation Réseaux, Programmation C.

Forme d'examen :

Contrôle Continue et Examen écrit.

Références bibliographique :

1.Réseaux, A. Tenenbaum, InterEditions
2.TCP/IP, Architecture, Protocoles et applications, D. Comer, InterEditions
3.La communication sous UNIX, J.M. Riflet, Mc Graw Hill
4.UNIX Network Programming - Networking APIs : Sockets and XTI, Volume 1, de W.Stevens Prentice Hall, 1997

Plan

- TCP/IP
- Internet Protocol (IP)
- Notion de port
- Concept de Client/Serveur
- User Datagram Protocol (UDP)
- Transmission Control Protocol (TCP)

Contrôles

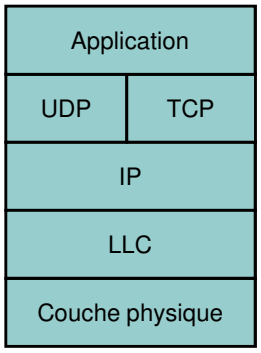
- Contrôles continus:
  - Rendus tous les TPs et tests : (0.25)
  - Rendu (Rapport, Démonstration) du mini projet : (0.25)
  - Ecrit : (0.5)

TCP/IP

- Standard de fait
- Normalisé par l'IETF
- TCP/IP est une suite de protocoles
- TCP/IP représente l'ensemble des règles de communication sur internet
- Fait abstraction des couches physique et liaison (notion de réseau d'interconnexion)
- Transmission par paquet
- Repose sur le fait que chaque machine du réseau possède une adresse IP

Pile de protocoles TCP/IP

- Pile à 5 couches
- Fonctionnalités essentielles de TCP/IP :
  - Le fractionnement des messages en paquets
  - L'utilisation d'un système d'adresses
  - L'acheminement des données sur le réseau (routage)
  - Le contrôle des erreurs de transmission de données

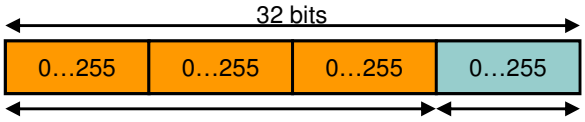


Internet Protocol
(IP)

- Permet l'élaboration et le transport des datagrammes IP (les paquets de données)
- Ne garantie pas leur livraison.
- Les datagrammes IP sont traités indépendamment les uns des autres
  - Ils peuvent donc prendre des chemins différents et arriver dans le désordre … s'ils arrivent !
- Le protocole IP détermine le destinataire du message grâce à son adresse **UNIQUE**

Adressage IP 1/7

- 32 bits (4 octets) pour coder 1 adresse
- Notation décimale pointée
  - aaa.bbb.ccc.ddd
  - Chaque valeur d'octet peut varier de 0 à 255
- La partie gauche de l'adresse désigne le réseau
- La partie droite de l'adresse désigne un hôte sur le réseau

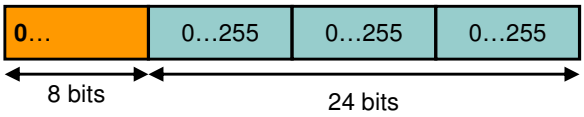


Adressage IP 2/7

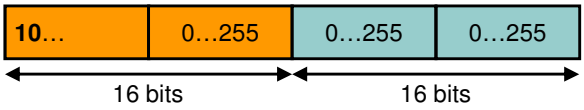
- Classes d'adresses :
  - Permet de faire varier la limite réseau←→hôte
  - Format d'adresse qui indique :
    - Le nombre de bits codant la partie réseau
    - Le nombre de bits codant la partie hôte
  - 4 grandes classes : A, B, C et D

Adressage IP 3/7

- Classe A
  - 8 bits réseaux, 24 bits hôte

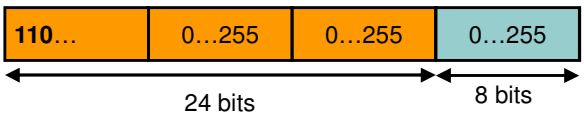


- Classe B
  - 16 bits réseaux, 16 bits hôte

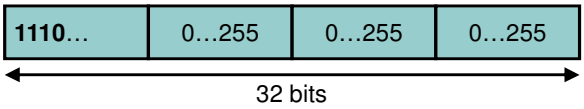


Adressage IP 4/7

- Classe C
  - 24 bits réseaux, 8 bits hôte

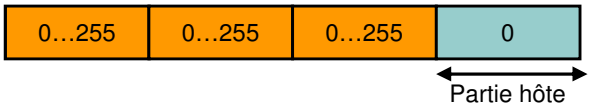


- Classe D
  - Réservée au communications multicast



Adressage IP 5/7

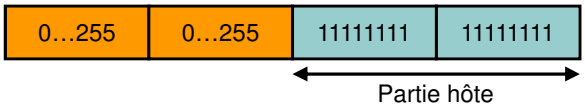
- Adresses particulières
  - Partie hôte à 0 indique une adresse de réseau :



  - Adresses **NON UTILISABLES** pour des hôtes
  - Exemple :
    - 141.123.0.0 ou 17.0.0.0

Adressage IP 6/7

- Adresses particulières
  - Partie hôte tout à 1 indique une adresse de diffusion (*broadcast*) :



  - Adresses **NON UTILISABLES** pour des hôtes
  - Exemple :
    - 1.255.255.255 ou 150.1.255.255 ou 200.45.70.255

Adressage IP 7/7

- Adresses particulières
  - 127.0.0.1 → Adresse de rebouclage (*loopback*)
  - 0.0.0.0 → Pas d'adresse IP (*DHCP*)
- Adresses privés (non reliées à Internet)
  - 10.x.y.z
  - De 172.16.0.0 à 172.31.255.255
  - De 192.168.0.0 à 192.168.255.255

Masque

- CIDR (*Classless Inter Domain Routing* RFC 1518, 1519)
- Masque: Indique les bits utilisés pour identifier le réseau

adresse IP	150.50.215.200	10010110.00110010.11010111.11001000
masque (21)	255.255.248.0	11111111.11111111.11111000.00000000
adresse réseau	150.50.208.0	10010110.00110010.11010 000.00000000
adresse machine	0.0.0.7.200	00000000.00000000.00000 111.11001000

Adresse IP : CIDR

- Adressage par classe :
  - utilisation inefficace de l'espace d'adressage
  - Ex : une adresse de classe B a assez de place pour pour 65K hôtes, même si il n'y a que 2K hôtes dans ce réseau
  - CIDR : Classless InterDomain Routing
  - La taille de la partie réseau est arbitraire
  - Format de l'adresse : a.b.c.d/x, où x est le # de bits dans la partie réseau de l'adresse

network part ← → host part

11001000 00010111 00010000 00000000

200.23.16.0/23

Notion de Port

- L'adresse IP identifie de manière unique un hôte sur un réseau MAIS elle n'identifie pas à quel programme les données sont destinées
- On utilise les ports pour différencier les différentes connexions simultanées sur une même machine
- Le couple <@IP, port> est unique
- Codé sur 16 bits
  - Varie de 1 à 65535
- Les numéros de port inférieurs à 1023 sont dits *well known*
  - Utilisés uniquement pour les services les plus connus
- Les numéros de port de 1024 à 49151 sont dits *réservés (ou ports utilisateurs)*
  - Utilisés uniquement pour les services et/ou applications
- Les numéros de port supérieurs sont libres !
  - Souvent appelés ports dynamiques ou privés
  - Disponible pour toute application et souvent employés à la demande. Ex: connexion d'un client web

Exemples de ports

No port	Mot-clé	Description
20	FTP-DATA	File Transfer [Default Data]
21	FTP	File Transfer [Control]
23	TELNET	Telnet
25	SMTP	Simple Mail Transfer
37	TIME	Time
42	NAMESERVER	Host Name Server
43	NICNAME	Who Is
53	DOMAIN	Domain Name Server
79	FINGER	Finger
80	HTTP	WWW
110	POP3	Post Office Protocol - Version 3
111	SUNRPC	SUN Remote Procedure Call

TCP : Transmission Control Protocol

- transport fiable de la technologie TCP/IP
  - fiabilité = illusion assurée par le service
  - découpage en segments
  - connexions bidirectionnelles et simultanées
- service en mode connecté
- garantie de remise de messages et d'ordonnancement

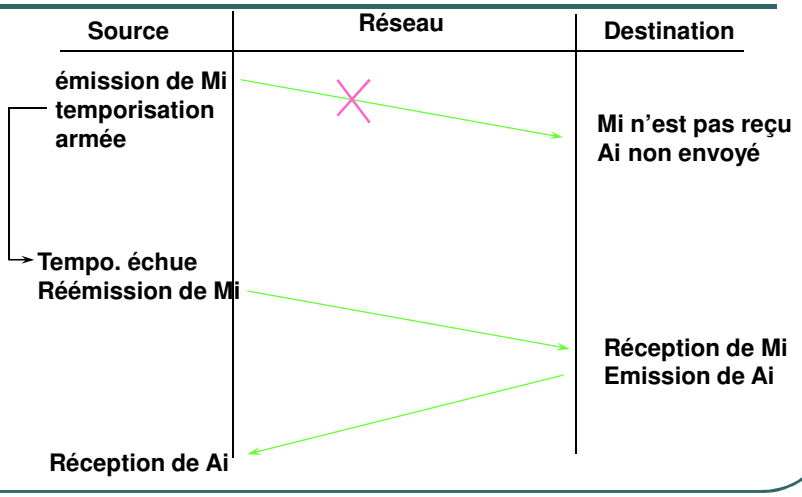
La connexion TCP

- une connexion de type circuit virtuel est établie
- connexion = une paire d'extrémités de connexion
- extrémité de connexion = couple (adresse IP, numéro port)
- Exemple de connexion : ((124.32.12.1, 1034), (19.24.67.2, 21))
- Une extrémité de connexion peut être partagée par plusieurs autres extrémités de connexions (multi-instanciation)
- La mise en oeuvre de la connexion se fait en deux étapes :
  - une application (extrémité) effectue une ouverture passive en indiquant qu'elle accepte une connexion entrante,
  - une autre application (extrémité) effectue une ouverture active pour demander l'établissement de la connexion.

Qualité de service

- Segmentation, contrôle de flux
  - les données transmises à TCP constituent un flot d'octets de longueur variable
  - TCP divise ce flot de données en segments en utilisant un mécanisme de fenêtrage
  - chaque segment est émis dans un datagramme IP
- Acquittement de messages
  - TCP garantit l'arrivée des messages, c'est-à-dire qu'en cas de perte, les deux extrémités sont prévenues
  - Ce concept repose sur les techniques d'acquittement de message : lorsqu'une source S émet un message Mi vers une destination D, S attend un acquittement Ai de D avant d'émettre le message suivant Mi+1.
  - Si l'acquittement Ai ne parvient pas à S, S considère au bout d'un certain temps que le message est perdu et réémet Mi

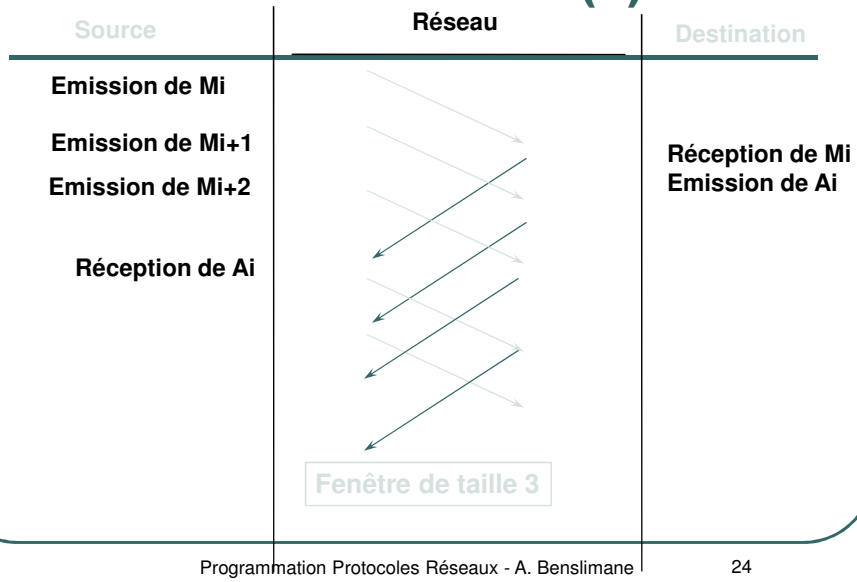
TCP : gestion des acquittements



La fenêtre

- La technique acquittement simple pénalise les performances puisqu'il faut attendre un acquittement avant d'émettre un nouveau message. La fenêtre améliore le rendement des réseaux.
- La technique : une fenêtre de taille T permet l'émission d'au plus T messages "*non acquittés*" avant de ne plus pouvoir émettre

Gestion de la fenêtre (1)

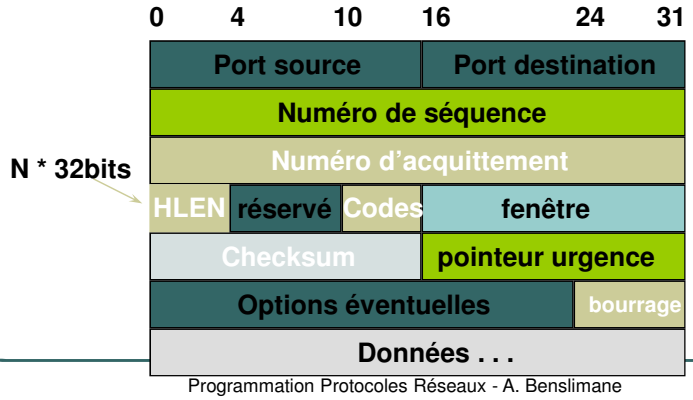


Gestion de la fenêtre (2)

- fenêtre coulissante permettant d'optimiser la bande passante
- permet également au destinataire de faire diminuer le débit de l'émetteur donc de gérer le contrôle de flux
- Le mécanisme de fenêtre mis en œuvre dans TCP opère au niveau de l'octet et non pas au niveau du segment ; il repose sur :
  - la numérotation séquentielle des octets de données,
  - la gestion de trois pointeurs par fenêtre

Format du segment TCP

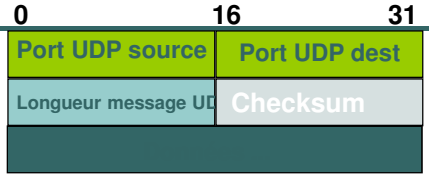
- Segment : unité de transfert du protocole TCP
  - pour établir les connexions
  - transférer les données et émettre des acquittements
  - fermer les connexions



UDP : User Datagram Protocol

- UDP : protocole de transport sans connexion
  - émission de messages applicatifs : sans établissement de connexion au préalable
  - l'arrivée des messages et l'ordonnancement ne sont pas garantis
- Identification du service : les ports
  - les adresses IP désignent les machines entre lesquelles les communications sont établies. Lorsqu'un processus désire entrer en communication avec un autre processus, il doit adresser le processus s'exécutant cette machine.
  - L'adressage de ce processus est effectué selon un concept abstrait indépendant du système d'exploitation :
    - les processus sont créés et détruits dynamiquement sur les machines, il faut pouvoir remplacer un processus par un autre sans que l'application distante ne s'en aperçoive,
    - il faut identifier les destinations selon les services offerts, sans connaître les processus qui les mettent en oeuvre,
    - un processus doit pouvoir assurer plusieurs services.

Le format des messages



Les ports source et destination contiennent les numéros de port utilisés par UDP pour démultiplexer les datagrammes destinés aux processus en attente de les recevoir. Le port source est facultatif (égal à zéro si non utilisé).

La longueur du message est exprimée en octets (8 au minimum) (en-tête + données), le champ de contrôle est optionnel (0 si non utilisé).

Les ports standards

- Certains ports sont réservés (*well-known ports*)

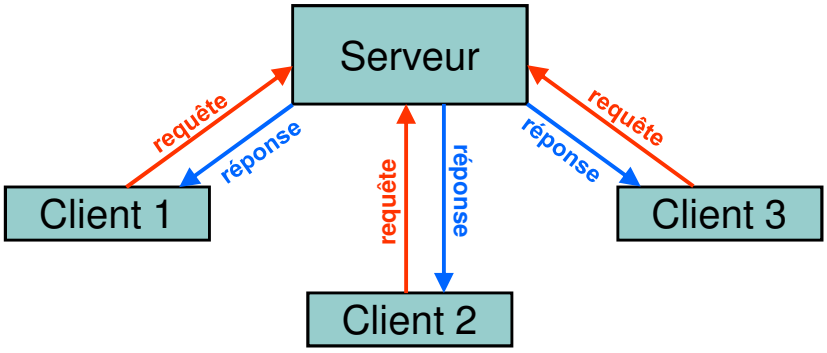
No port	Mot-clé	Description
7	ECHO	Echo
11	USERS	Active Users
13	DAYTIME	Daytime
37	TIME	Time
42	NAMESERVER	Host Name Server
53	DOMAIN	Domain Name Server
67	BOOTPS	Boot protocol server
68	BOOTPC	Boot protocol client
69	TFTP	Trivial File Transfer protocol
123	NTP	Network Time Protocol
161	SNMP	Simple Network Management prot.

- D'autres numéros de port (non réservés) peuvent être alloués dynamiquement aux applications.

Paradigme Client/Serveur

- Un service est un programme fournissant des données :
  - heure, des fichiers, une connexion...
- Un serveur est une entité (logique et éventuellement physique) qui rend un service
  - On parle ainsi de serveur web, serveur mail, serveur X …
- Les machines clientes contactent un serveur pour obtenir des services
  - On parle de client web, client mail, client FTP…
- Dans un environnement purement Client/Serveur, les ordinateurs du réseau (les clients) ne peuvent voir que le serveur

Architecture Client/Serveur



Avantages et Inconvénients du Client/Serveur

- Avantages :
  - Ressources centralisées
    - Le serveur peut gérer toutes les ressources communes aux les utilisateurs. Cela évite les problèmes de redondance d'information.
  - Meilleure sécurité
    - Les points vulnérables se limitent aux serveurs.
  - Administration facilitée
    - Seul le serveur a besoin d'une réelle administration
  - Évolutivité
    - Il est possible d'ajouter/retirer des clients facilement
- Inconvénients :
  - LE maillon faible: Tout repose sur le serveur. S'il tombe en panne, le réseau peut être paralysé !

User Datagram Protocol
(UDP)

- Protocole se basant sur IP
- **NON** orienté connexion
- **NON** fiable
- Fonctionnement par datagrammes
- Ne fait pas grand chose de plus qu'IP :
  - Ajout de la notion de port à IP
  - Fait l'interface entre les applications et IP
- Bien adapté aux applications Client/Serveur
  - Requête → réponse rapide ! Seulement 2 messages !
- Exemples de services basés sur UDP :
  - DNS, NFS, SNMP…

Transmission Control Protocol
(TCP)

- Protocole de Contrôle de Transmission
- Protocole se basant sur IP
- Caractéristiques :
  - Orienté connexion
  - Fiable (gestion des pertes et des erreurs)
  - Fonctionnement par paquets
  - Gère le séquencement des paquets
  - Contrôle de flux
  - Formate les données en segments de longueur variable pour IP
  - Gère les ports
  - TCP gère l'initialisation et la fin des communications

Fonctionnement simplifié de TCP

- Début de connexion :
  - Les ports TCP doivent être ouverts
  - L'application sur le serveur est passive
    - Elle « écoute », en attente d'une connexion
  - L'application sur le client fait une requête de connexion sur le serveur dont l'application est en ouverture passive (port de l'application sur le serveur en écoute.
- Fin de connexion :
  - Le client et le serveur peuvent demander à mettre fin à la connexion
  - Le demandeur de fin indique qu'il ferme la connexion

Exemples d'applications utilisant TCP

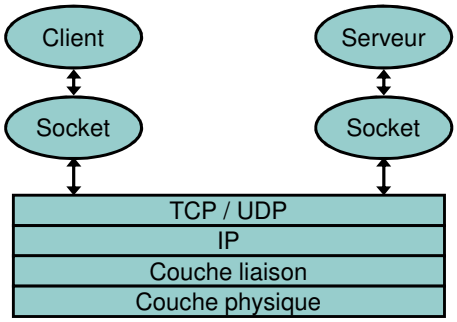
- FTP
- Telnet
- SMTP
- POP
- IMAP
- …

Notion d'API

- API = Application Programming Interface
- Ensemble de primitives de programmation

Définition SOCKET

- Le terme Socket désigne à la fois une bibliothèque d'interface réseau et l'extrémité d'un canal de communication par lequel un processus peut émettre ou recevoir des données



Domaine d'une Socket 1/3

- Le domaine spécifie le format des adresses qui pourront être données à la socket et les différents protocoles supportés pour les communications via les sockets de ce domaine
- Exemple de domaines :
  - AF_UNIX : pour les sockets UNIX
  - AF_INET : pour les sockets Internet (TCP/IP)
  - X25 : pour le réseau X25
  - …

Domaine d'une Socket 2/3

- Domaine UNIX (AF_UNIX) :
  - Ces sockets sont destinées à la communication locale.
  - Une socket du domaine UNIX apparaît après nommage dans les résultats fournis par la commande `ls`
  - On peut la détruire via la commande ls
  - La structure d'adresse est `sockaddr_un`

Domaine d'une Socket 3/3

- Domaine Internet (AF_INET) :
  - Ces sockets sont destinées aux communications via Internet
  - Les sockets AF_INET obéissent aux règles d'adressage IP
  - La structure d'adresse est `sockaddr_in`

Type d'une socket

- Le type définit les propriétés des communications
  - **`SOCK_DGRAM`**
    - Supporte le mode datagramme
      - Communications non fiables (pertes, déséquencement)
      - Mode non connecté (UDP)
      - Limite de taille (pas de découpage des messages)
  - **`SOCK_STREAM`**
    - Supporte le mode paquet (flot d'octets)
      - Communication fiable
      - Gestion du séquencement
      - Pas de duplication
      - Mode connecté (TCP)
  - **`SOCK_RAW`**
    - Permet l'accès aux protocoles de plus bas niveau (IP par exemple)
    - Ou permet d'implanter de nouveaux protocoles (réservé au super-utilisateur)

Adressage Part. 1

- Utilisation de la structure générique `<sys/socket.h>`
- Domaine AF_INET
  - Les processus spécifient le transport utilisé (TCP ou UDP) via le type de la socket
  - Le serveur associe son service à une adresse et un numéro de port et se met en attente d'une connexion
  - Le client envoie ses requêtes en précisant l'adresse internet du serveur et le numéro du port
- Domaine AF_UNIX
  - Les échanges entre le client et le serveur s'effectuent à travers un fichier UNIX de type socket `<sys/un.h>`

Ordre 1/3

- **La représentation des adresses IP peuvent être différente (ordre des octets) entre :**
  - **Leur représentation réseau**
  - **La représentation machine**
- Les fonctions de conversions sont dans `<sys/types.h>` et `<netinet/in.h>`

Ordre 2/3

- Représentation réseau → locale :
  - `u_long ntohl(x)`
    - Retourne la représentation locale en entier **LONG**
    - Paramètre :
      - `int x`
        - Représentation réseau **ENTIER LONG**
  - `u_short ntohs(x)`
    - Retourne la représentation locale en entier **COURT**
    - Paramètre :
      - `int x`
        - Représentation réseau **ENTIER COURT**

Ordre 3/3

- Représentation locale → réseau :
  - `u_long htonl(x)`
    - Retourne la représentation réseau en entier **LONG**
    - Paramètre :
      - `int x`
        - Représentation locale **ENTIER LONG**
  - `u_short htons(x)`
    - Retourne la représentation réseau en entier **COURT**
    - Paramètre :
      - `int x`
        - Représentation locale **ENTIER COURT**

Conversions 1/3

- Conversions de chaines en adresses :
  - Les fonctions se trouvent dans `<sys/types.h>`, `<netinet/in.h>` et `<arpa.inet.h>`
  - `int inet_aton(chaine,adresse)`
    - Convertie une chaine de caractères (ex: 212.23.4.5) en adresse
    - Vérifie si l'adresse obtenue est correcte
    - Retourne 1 si OK, 0 sinon
    - Paramètres :
      - `char *chaine`
        - Pointeur vers la chaine à convertir
      - `struct in_addr adresse`
        - Structure d'adresse recevant le résultat de la conversion

Conversions 2/3

- `u_long inet_addr(adresse)`
  - Convertie une chaine de caractère ASCII en adresse au format long
  - Pas de vérification de l'adresse convertie
  - Retourne un long contenant l'adresse convertie, `INADDR_NONE` sinon
  - Paramètre :
    - `char *adresse`
      - Pointeur vers la chaine ASCII à convertir

Conversions 3/3

- `char *inet_ntoa(adresse)`
  - Converti une adresse en chaine de caractères
  - Retourne un pointeur non nul si OK. Ce pointeur renvoie vers la chaine de caractères contenant le résultat de la conversion
  - Paramètre :
    - `struct in_addr adresse`
      - Structure d'adresse à convertir

Adresses Part. 2 1/6

- L'utilisation de socket demande l'utilisation de structure d'adresses
- Se trouvent dans `<netinet/in.h>`
- Commencent par `sockaddr`
  - `sockaddr_in` pour IPv4
  - `sockaddr_un` pour sockets UNIX
  - `sock_addr_in6` pour IPv6

Adresses Part. 2 2/6

- Structure d'adresse IPv4 :
  - `struct sockaddr_in`
    - `uint8t sin_len`
      - Longueur de la structure
    - `sa_family_t sin_family`
      - Doit être positionné à **`AF_INET`**
    - `in_port_t sin_port`
      - Numéro de port TCP ou UDP (**ordre réseau**)
      - Si valeur de port = 0, le système choisit un port
    - `struct in_addr sin_addr`
      - Adresse 32 bits IPv4 (**ordre réseau**)
    - `char sin_zero[8]`
      - Non utilisé mais doit être initialisé à 0

Adresses Part. 2 3/6

- `struct in_addr`
  - `in_addr_t s_addr`
    - Adresse IPv4 32 bits
    - **ORDRE RESEAU**
      - **utiliser les fonctions de conversions**
- Valeurs particulières :
  - `INADDR_ANY` : spécifie toutes les adresses de la machine
  - `INADDR_LOOPBACK` : spécifie l'adresse de rebouclage

Adresses Part. 2 4/6

- Structure d'adresse générique :
  - Indépendant du protocole
  - `struct sockaddr`
    - `uint8t sin_len`
      - Longueur de la structure
    - `sa_family_t sin_family`
      - Doit être positionné à **`AF_INET`** pour les sockets internet
    - `char sa_data[14]`
      - Données à passer au protocole
      - Dépend du protocole

Adresses Part. 2 5/6

- Structure d'adresse UNIX :
  - `struct sockaddr_un`
    - `uint8t sun_len`
      - Longueur de la structure
    - `sa_family_t sun_family`
      - Doit être positionné à **`AF_LOCAL`**
    - `char sun_path[104]`
      - Chemin où la socket doit être positionnée
      - Doit terminer par `null`
- Valeurs particulières :
  - `sun_path[0] = 0` ←→ `INADDR_ANY`

Adresses Part. 2 6/6

- Les fonctions de manipulation de socket utilisent `sockaddr`
- Pour utiliser une autre forme (notamment `sockaddr_in`) :
  - *Caster* la nouvelle forme en `sockaddr`
  - Exemple :

```
struct sockaddr_in mon_adresse;
/* … */
bind(chaussette,(struct sockaddr *)&mon_adresse
                ,sizeof(mon_adresse))
```


Routines auxiliaires 1/8

- Nom de la machine locale :
  - `int gethostname(nom, lg)`
    - Met le nom de la machine dans nom
    - Se trouve dans `<netdb.h>`
    - Retourne `0` si ok, `-1` sinon
    - Paramètres :
      - `char *nom`
        - Pointeur sur la chaine de caractères qui reçoit le nom
      - `int lg`
        - taille de la zone réservée

Routines auxiliaires 2/8

- Caractéristiques d'un machine :
  - Se trouvent dans le fichier `/etc/hosts`
  - `struct hostent *gethostbyname(nom)`
    - Si on connaît le nom de la machine, on peut récupérer la structure `hostent` qui contient ses caractéristiques
    - Se trouve dans `<netdb.h>`
    - Retourne un pointeur non nul si ok
    - Paramètre :
      - `char *nom`
  - `struct hostent *gethostbyaddr(adresse, lg, type)`
    - Si on connaît l'adresse de la machine, on peut récupérer la structure `hostent` qui contient ses caractéristiques
    - Se trouve dans `<netdb.h>`
    - Retourne un pointeur non nul si ok
    - Paramètres :
      - `char *adresse` : adresse de la machine
      - `int lg` : longueur de la structure
      - `int type` : type de protocole

Routines auxiliaires 3/8

- Structure `hostent` :
  - `char *h_name`
    - Nom canonique de l'hôte
  - `char **h_aliases`
    - Pointeur vers le tableau de pointeurs vers les alias de nom
  - `int h_addrtype`
    - Type d'adresse (`AF_INET`, `AF_INET6`)
  - `int h_length`
    - Longueur de l'adresse (4 pour IPv4)
  - `char **h_addr_list`
    - Pointeur vers le tableau de pointeurs vers les autres adresses de la machine

Routines auxiliaires 4/8

- Caractéristiques d'un service sur une machine :
  - `struct servent *getservbyname(nom, proto)`
    - Si on connaît le nom du service, on peut récupérer la structure servent qui contient ses caractéristiques
    - Se trouve dans `<netdb.h>`
    - Retourne un pointeur non nul si ok
    - Paramètres :
      - `char *nom` = Nom du service
      - `char *proto` = Nom du protocole (udp, tcp)
  - `struct servent *getservbyport(port, proto)`
    - Si on connaît le port du service, on peut récupérer la structure servent qui contient ses caractéristiques
    - Se trouve dans `<netdb.h>`
    - Retourne un pointeur non nul si ok
    - Paramètres :
      - `int port`
      - `char *proto`

Routines auxiliaires 5/8

- Structure `servent` :
  - `char *s_name`
    - Nom officiel du service
  - `char **s_aliases`
    - Pointeur vers le tableau de pointeurs vers les alias de nom
  - `int s_port`
    - Numéro de port (**ordre réseau**)
  - `char **s_proto`
    - Pointeur vers le tableau de pointeurs vers les noms des protocoles à utiliser pour ce service

Routines auxiliaires 6/8

- Récupération de l'adresse et du numéro de port d'une socket
- Sur la socket locale :
  - `int getsockname(socket, adresse, lg)`
    - Se trouve dans `<sys/socket.h>`
    - Retourne `0` si ok, `-1` sinon
    - Paramètres :
      - `int socket`
        - Descripteur de la socket
      - `struct sockaddr *adresse`
        - Pointeur sur la structure adresse devant récupérer l'adresse et le numéro de port
      - `Socklen_t *lg`
        - Pointeur sur la taille
- Sur la socket distante :
  - `int getpeername(socket, adresse, lg)`
    - Se trouve dans `<sys/socket.h>`
    - Retourne `0` si ok, `-1` sinon
    - Paramètres identiques à `getsockname`

Routines auxiliaires 7/8

- `void bcopy(b1,b2,lg)`
  - Copie `lg` octets de `b1` (chaine source) vers `b2` (chaine destination)
  - Paramètres :
    - `char *b1`
    - `char *b2`
    - `int lg`
- `void bzero(adresse,lg)`
  - Initialise la chaine pointée par `adresse` par `lg` zéros
  - <u>Utilisé pour initialiser les structures d'adresses</u>
  - Paramètres :
    - `char *adresse`
    - `int lg`

Routines auxiliaires 8/8

- `void bcmp(b1,b2,lg)`
  - Compare `lg` octets de `b1` (chaine source) vers `b2` (chaine destination)
  - Paramètres :
    - `char *b1`
    - `char *b2`
    - `int lg`

Fichiers d'inclusion

AF_INET	AF_UNIX
<stdio.h>	<stdio.h>
<sys/types.h>	<sys/types.h>
<sys/socket.h>	<sys/socket.h>
<netinet/in.h>	**<sys/un.h>**
<netdb.h>	<netdb.h>
<errno.h>	**#define PATH "tmp/nomfich"**
#define PORT num_port	
socin.h	socun.h

Du bon usage des sockets 1/2

1. Définition des variables
   - Descripteurs des sockets, structures d'adresses, ports …
2. Préparation des adresses
   - Affectation des différents champs
3. Ouverture de la socket

Du bon usage des sockets 2/2

- Préparation des adresses :
  1. Remise à zéro de la structure
     - `bzero`
  2. Affectation des différents champs
     - `sin_family`, `sin_addr`, `sin_port`, `sin_len`
       - ATTENTION à l'ordre (`htons`…)

Création de Sockets

- Fonctions de manipulation de sockets dans `<sys/socket.h>`
- `int socket(domaine, type, protocole)`
  - Crée une socket
  - Retourne le descripteur de la socket (`int`) si ok , `-1` sinon
  - Paramètres :
    - `int domaine`
      - Domaine d'application de la socket (`AF_INET`, `AF_LOCAL`...)
    - `int type`
      - Mode de transmission (`SOCK_DGRAM`, `SOCK_STREAM` ou `SOCK_RAW`)
    - `int protocole`
      - Choix du protocole. La valeur doit être `0`, sauf pour des sockets RAW

Liaison d'une socket à une adresse

- `int bind(socket, adresse, lg)`
  - Attache une socket à une adresse
  - Retourne `0` si ok, `-1` sinon
  - Paramètres :
    - `int socket`
      - Descripteur d'une socket existante
    - `struct sockaddr *adresse`
      - Structure d'adresse
    - `int lg`
      - Taille de la structure d'adresse

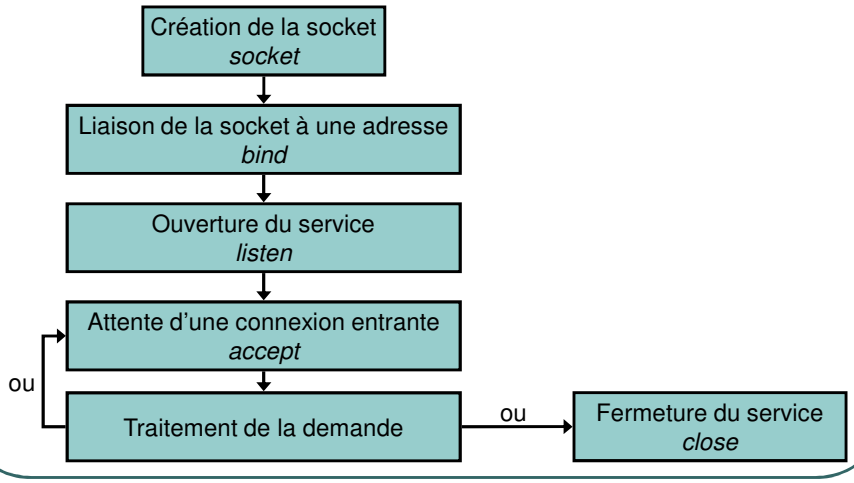
Fin de connexion

- `int close(socket)`
  - Termine la connexion en cours
  - Peut être appelé par le serveur ou le client
  - Retourne `0` si ok, `-1` sinon
  - Paramètre :
    - `int socket`
      - Descripteur de la socket

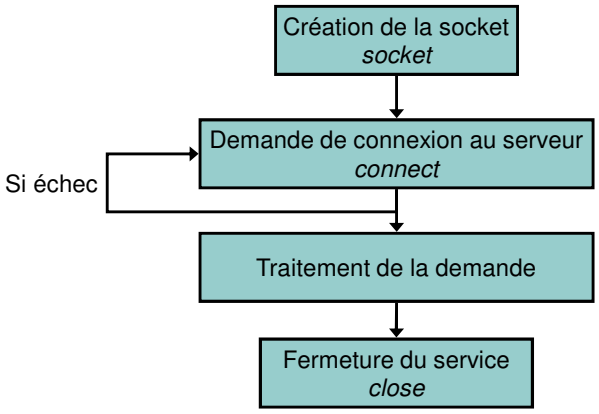
Sockets en mode Flot

- Connexion du point de vue du serveur
- Connexion du point de vue du client
- Opérations de manipulation des sockets en mode flot
- Opération de manipulation du flot
  - Les sockets peuvent être vus comme des flux dans des fichiers. On peut donc utiliser des opérations standard.

Socket en mode flot (serveur)



Socket en mode flot (client)



Connexion à une socket

- Employé uniquement par le **client**
- `int connect(socket, adresse, lg)`
  - Se connecte à une socket
  - Retourne `0` si ok, `-1` sinon
  - Paramètres :
    - `int socket`
      - Descripteur de la socket (résultat de la fonction socket)
    - `struct sockaddr *adresse`
      - Structure d'adresse indiquant à la socket où se connecter
    - `int lg`
      - Taille de la structure d'adresse `adresse`

Écoute d'une Socket

- Employé uniquement par le **serveur**
- S'emploie APRES le `bind`
- `int listen(socket, backlog)`
  - Transforme la socket non connectée en socket d'écoute en attente d'une connexion entrante
  - Retourne `0` si ok, `-1` sinon
  - Paramètres :
    - `int socket`
      - Descripteur de la socket
    - `int backlog`
      - Taille de la file d'attente des connexions entrantes
        - ie. Combien de connexions simultanées puis-je recevoir ?

Acceptation d'une connexion entrante sur une socket

- Employée uniquement par le **serveur**
- `int accept(socket, adresse, lg)`
  - Bloque un processus en attente d'une connexion entrante
  - Retourne le descripteur du flux entrant si ok, `-1` sinon
  - Paramètres :
    - `int socket`
      - Descripteur de la socket d'écoute
    - `struct sockaddr *adresse`
      - Positionné par la fonction
      - Structure d'adresse du client qui se connecte
    - `socklen_t *lg`
      - Positionné par la fonction
      - Taille de la structure d'adresse du client (`adresse`)

Lecture dans une socket

- Ce n'est pas le seul moyen de lire dans une socket
- `int read(socket, tampon, nb)`
  - Lit des données dans une socket connectée
  - Retourne le nombre d'éléments lus si ok, `-1` sinon
  - Paramètres :
    - `int socket`
      - Descripteur de la socket
    - `Tampon`
      - Positionné par la fonction
      - Tampon où doivent être écrites les données reçues
    - `nb`
      - Nombre d'éléments du tampon à lire

Écriture dans une socket

- Ce n'est pas le seul moyen d'écrire dans une socket
- `int write(socket, tampon, nb)`
  - Écrit des données dans une socket connectée
  - Retourne le nombre d'éléments envoyés si ok, `-1` sinon
  - Paramètres :
    - `int socket`
      - Descripteur de la socket
    - `Tampon`
      - Tampon où doivent être lues les données à envoyer
    - `nb`
      - Nombre d'éléments du tampon à envoyer

Exemple de serveur TCP

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>

int main (int argc, char *argv[])
{
 int client_socket, ma_socket;
 char buffer[512];
 struct sockaddr_in mon_address, client_address;
 int mon_address_longueur, lg;

 bzero(&mon_address,sizeof(mon_address));
 mon_address.sin_port = htons(30000);
 mon_address.sin_family = AF_INET;
 mon_address.sin_addr.s_addr = htonl(INADDR_ANY);

/* creation de socket */
if ((ma_socket = socket(AF_INET,SOCK_STREAM,0))== -1)
{  printf("pb de creation de la socket\n"); exit(0); }

/* bind serveur - socket */
bind(ma_socket,(struct sockaddr *)&mon_address,
      sizeof(mon_address));

/* ecoute sur la socket */
listen(ma_socket,5);

/* accept la connexion */
mon_address_longueur = sizeof(client_address);
while(1)
 {
  client_socket = accept(ma_socket,
                         (struct sockaddr *)&client_address,
                         &mon_address_longueur
                         );

  lg = read(client_socket,buffer, 512);
  printf("le serveur a recu: %s\n",buffer);

  sprintf(buffer,"%s du serveur",buffer);

  write(client_socket,buffer, 512);
  close(client_socket);
 }
close(ma_socket);
}
```

Exemple de client TCP

```
#include <stdio.h>
#include <errno.h>
#include <netdb.h>
#include <netinet/in.h>
#include <sys/socket.h>

#define SERVEURNAME "127.0.0.1"

void main ( void )
{
int to_server_socket = -1;
char *server_name = SERVEURNAME;
struct sockaddr_in serverSockAddr;
struct hostent *serverHostEnt;
long hostAddr, status;
char buffer[512];
bzero(&serverSockAddr,sizeof(serverSockAddr));
hostAddr = inet_addr(SERVEURNAME);
if ( (long)hostAddr != (long)-1)
        bcopy(&hostAddr,
                &serverSockAddr.sin_addr,sizeof(hostAddr));
else
{
  serverHostEnt = gethostbyname(SERVEURNAME);
  if (serverHostEnt == NULL)
  { printf("Le gethost coince\n"); exit(0); }
  bcopy(serverHostEnt->h_addr,
          &serverSockAddr.sin_addr,
          serverHostEnt->h_length);
}
serverSockAddr.sin_port = htons(30000);
serverSockAddr.sin_family = AF_INET;

/* creation de la socket */
if ( (to_server_socket = socket(AF_INET,SOCK_STREAM,0)) < 0)
{ printf("Pb de creation du socket client\n"); exit(0); }
/* requete de connexion */
if(connect( to_server_socket,(struct sockaddr *)&serverSockAddr,
                sizeof(serverSockAddr)) < 0 )
  { printf("Pb dans la demande de connexion\n");  exit(0); }

/* envoie de donnees et reception */
write(to_server_socket,"coucou",6);
read(to_server_socket,buffer,512);
printf(buffer);
/* fermeture de la connection */
close(to_server_socket);
}
```

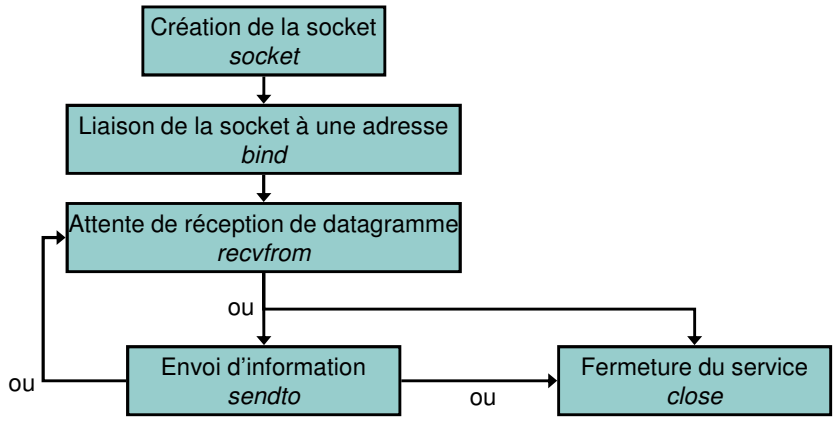

EXERCICE :

- Écrire un programme client permettant d'interroger un serveur *daytime* dont l'adresse est fournie en paramètre
- Écrire un programme serveur de type *daytime*

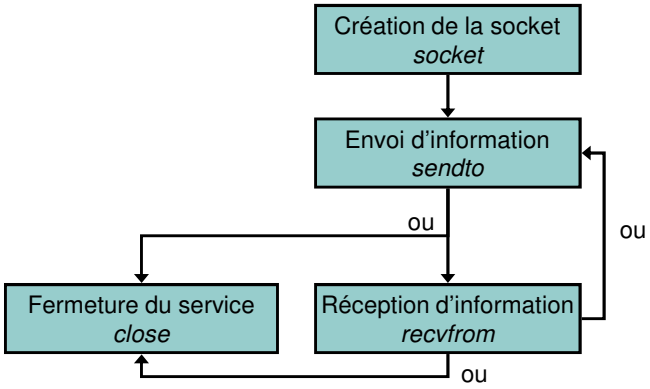
Socket en mode Datagramme

- Point de vue du serveur
- Point de vue du client
- Opérations de manipulation des sockets en mode datagramme

Socket en mode datagramme (serveur)



Socket en mode datagramme (client)



Réception d'un datagramme

- Fonctions dans `<sys/socket.h>`
- `int recvfrom(socket, tampon, taille, drapeau, adresse, lg)`
  - Bloque un processus en attente de réception d'un datagramme
  - Retourne le nombre d'octet effectivement lus si ok, `-1` sinon
  - Paramètres :
    - `int socket`
      - Descripteur de la socket (résultat de la fonction socket)
    - `void *tampon`
      - Pointeur vers le tampon où les données doivent être écrites
    - `size_t taille`
      - Nombre d'octet à recevoir
    - `int drapeau`
      - Permet d'indiquer des options. Ce champ est généralement mis à 0
    - `struct sockaddr *adresse`
      - Structure d'adresse du client qui a envoyé le datagramme reçu
    - `int lg`
      - Taille de la structure d'adresse `adresse`

Réception d'un datagramme

- `int sendto(socket, tampon, taille, drapeau, adresse, lg)`
  - Envoi un datagramme
  - Retourne le nombre d'octet effectivement envoyés si ok, `-1` sinon
  - Paramètres :
    - `int socket`
      - Descripteur de la socket (résultat de la fonction socket)
    - `void *tampon`
      - Pointeur vers le tampon à partir duquel les données seront lues
    - `size_t taille`
      - Nombre d'octet à lire
    - `int drapeau`
      - Permet d'indiquer des options. Ce champ est généralement mis à 0
    - `struct sockaddr *adresse`
      - Structure d'adresse indiquant la destination du datagramme
    - `int lg`
      - Taille de la structure `adresse`

Exemple de serveur utilisant le mode datagramme

```
#include <sys/types.h>
#include <sys/socket.h>
#include <stdio.h>
#include <netinet/in.h>
#include <netdb.h>
#include <errno.h>
#include <arpa/inet.h>

#define MSGSIZE 500

int main(int argc, char **argv)
{
  int sockfd, cc, addr_in_size;
  u_short portnum = 12345;
  struct sockaddr_in my_addr, from;
  char msg[MSGSIZE];
  u_long fromaddr;
  addr_in_size = sizeof(struct sockaddr_in);

  bzero(&my_addr,addr_in_size);
  my_addr->sin_family = AF_INET;
  my_addr->sin_addr.s_addr = htonl(INADDR_ANY);
  my_addr->sin_port = htons(portnum);

 if((sockfd = socket (AF_INET, SOCK_DGRAM, 0)) < 0){
    fprintf(stderr,"Error %d in socket: %s\n",errno,sys_errlist[errno]);
    exit(errno);
 };

 if(bind(sockfd, (struct sockaddr *)my_addr, addr_in_size) < 0){
   fprintf(stderr,"Error %d in bind: %s\n",errno,sys_errlist[errno]);
   if(errno != EADDRINUSE) exit(errno);
 };

 fprintf(stdout,"Ready to receive\n");

 while(1){
   if((cc = recvfrom (sockfd,msg,MSGSIZE,0,(struct sockaddr *)from,
         &addr_in_size)) == -1){
     fprintf(stderr,"Error %d in recvfrom: %s\n",
       errno,sys_errlist[errno]);
     exit(errno);
   };
   fromaddr = from->sin_addr.s_addr;
   msg[cc] = '\0';
   fprintf(stdout,"From %s port %d: %s\n",
     (gethostbyaddr((char *)&fromaddr,
       sizeof(fromaddr),
       AF_INET))->h_name,
     from->sin_port,msg);
 }
}
```

Exemple de client utilisant le mode datagramme

```
#include <stdio.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <arpa/inet.h>

#define MSGSIZE 500

int main(int argc,char **argv)
{
  int sockfd, ok, addr_in_size;
  u_short portnum = 12345;
  struct sockaddr_in to;
  struct hostent *toinfo;
  char msg[MSGSIZE];
  addr_in_size = sizeof(struct sockaddr_in);

  char *htoname = "127.0.0.1";
  u_long toaddr;

   if((toinfo = gethostbyname(htoname)) == NULL){
    fprintf(stderr,"Error %d in gethostbyname: %s\n",
      errno,sys_errlist[errno]);
    exit(errno);
  };
  toaddr = *(u_long *)toinfo->h_addr_list[0];
memset((char *)to,(char)0,addr_in_size);

  to->sin_family = AF_INET;
  to->sin_addr.s_addr = toaddr;
  to->sin_port = (htons)portnum;

  if((sockfd = socket (AF_INET, SOCK_DGRAM, 0)) == -1){
    fprintf(stderr,"Error %d in socket: %s\n",errno,sys_errlist[errno]);
    exit(errno);
  };

  ok = 1;
  while(ok){
    fprintf(stdout,"Message: ");
    scanf("%s",msg);
    fprintf(stdout,"\n");
    ok = (msg[0] != '0');

    if(sendto(sockfd,msg,strlen(msg),0,(struct sockaddr *)to,
        addr_in_size) == -1){
      fprintf(stderr,"Error %d in sendto: %s\n",errno,sys_errlist[errno]);
      exit(errno);
    }
  }
}
```


EXERCICE :

- Écrire un programme client permettant d'interroger un serveur *daytime* dont l'adresse est fournie en paramètre
- Écrire un programme serveur de type *daytime*

Superviseur Internet

- Système linux: il peut y avoir plusieurs (programmes) appelés démons en existence et seulement en attente de requêtes, tels que ftp, telnet, tftp, remote login, remote shell, etc.
- Cela existait avant la version BSD 4.3 : un programme pour chaque service est en execution
- /etc/rc startup
- >> BSD 4.3 => superserveur Internet: processus inetd
- C'est un démon utilisé en tcp ou udp; qui va s'occuper au moment venu de tous les services à rendre!
- => /etc/inetd.conf : dans ce fichier, le démon inetd va lire les services qu'il va lancer! Est organisé en plusieurs lignes; chaque ligne sera dédié à un service
  - nom service: /etc/ services
  - Type de socket: stream ou datagram
  - Protocole:/etc/protocols : tcp soit udp
  - Flag d'attente: wait ou en nowait
  - Nom login: /etc/password => root
  - Programme serveur: chemin complet à exec
  - Arguments du service: 5 arguments.
- Exp: ftp stream tcp nowait root /etc/ftpd ftpd

Blocage ou non blocage des fonctions sockets

- Les fonctions rcv, recvfrom accept, etc. sont des fonctions bloquantes
- Pour rendre une opération non bloquante:
- => fcntl(sock,F_SETFL, FNDELAY|fcntl(sock, F_GETFL,0)) /* fcntl.h
- Ou ioctl /sys/ioctl.h
- int on=1,
- ioctl(sock, FIONBIO, &on)
- Le fait de rendre les sockets non bloquantes, le systèmes me retourne des erreurs lorsque je manipule les sockets:
- Les fonctionnements des opérations sont les suivants:
- - accept(): revient immédiatement si aucune demande de connexion n'est en cours avec l'erreur EWOULDBLOCK
- - connect(): revient immédiatement avec l'erreur EINPROGRESS
- recv(), read(), recvfrom() retournent -1 (FIONBIO ou FNDELAY) l'erreur est positionnée à EWOULDBLOCK
- - send(), write() et sendto() //
- => rendre la socket bloquante: on=0; ioctl(sock, FIONBIO, &on) ou fcntl(sock, F_SETFL, ~ FNDELAY&fcntl(sock, F_GETFL,0))

Prise en compte des messages urgents

MSG_OOB

- TCP prévoit le transport des caractères dont l'arrivé est notifié au processus destinataire afin qu'il puisse les prendre en compte immédiatement
- La réalisation sous unix se fait avec la notion de données OOB (Out of Band) et l'asynchronisme est obtenu par le signal SIGURG
- L'émission: réalisée sur une socket de type SOCK_STREAM (TCP) avec la primitive send et l'option MSG_OOB: dans l'implémentation unix, un seul caractère urgent est conservé
- Les n-1 caractères envoyés sont desCc=send(sock, « bonjour », 7, MSG_OOB):
- caractères normaux et le nème c'est le caractère urgent; a, b et c sont normaux et seul d est urgent

MSG_OOB

- La réception: la réception est faite avec la fonction recv et l'option MSG_OOB
  - Il n'est pas nécessaire d'être positionné sur le caractère urgent pour le lire
  - recv(sock, chaine, sizeof(chaine), 0) => buter contre la marque repérant le caractère urgent
  - Recv(sock, ch, 1, MSG_OOB)

MSG_OOB

- Recherche de la marque d'un caractère urgent
  - Il est possible de tester au moyen de la primitive ioctl (système) et l'opération SIOCATMARK si la position courante dans le flot de caractères correspond à un caractère OOB

…

int rep;

ioctl(sock, SIOCATMARK, &rep): fonction système

=> rep=1 => marque atteinte

=➔ écrire une fonction de vidage de tous les caractères précédant la marque .

MSG_OOB

```
int vider (sock, curg)
int sock;
char *curg;   /* sock=vhflkvzefklbjzemkjbmzejbmX
{
int rep;
static char buf[TMAX];
while (1) {
  if (ioctl(sock,SIOCATMARK, &rep)==-1)
                              FATAL("ioctl");
  if (rep==1) break;   /* la marque est atteinte */
  /*sinon la marque n'est pas atteinte; on lit au plus TMAX caractères*/
 recv(sock, buf, sizeof(buf),0);
}
if (recv(sock,curg,1,MSG_OOB)==-1)   FATAL("MSG_OOB");
                /*lecture du car urg. */
Return(0);
}
```

MSG_OOB

- La devinette du mot caché: caractères urgents
  - Ecrire une application distribuée constituée d'un serveur et d'un client.
  - Côté client:
  - Il dispose d'un mot caché, par exemple, « bonjour »

  et l'envoi dilué dans des flots d'octets (plusieurs messages de caractères) au serveur.

  : ghqlhgklgkgghkghkb

  : Elvzefklvkolfnkznkncvkn

  : Rheyjetyjrukyuktyikjldzhflzfzjfkazjo……..uejfhelhfklruigfuhf

MSG_OOB

- La devinette du mot caché: caractères urgents
  - Ecrire une application distribuée constituée d'un serveur et d'un client:
  - Côté serveur:
  - Il va recevoir le flot et en même temps il va détecter l'arrivé de caractères urgents

  : ghqlhgklgkgghkghkb

  : Elvzefklvkolfnkznkncvkn

  : Rheyjetyjrukyuktyikjldzhflzfzjfkazjo……..uejfhelhfklruigfuhf

- Client:
- Envoyer la taille du mot caché au serveur
- While (taille du mot caché)
  - Lecture d'une chaine de caractères
  - Envoie de la chaine
  - Envoie d'un caractère urgent
- Serveur:
- Dan le main:
  - Associer le signal SIGURG à la socket au PID du processus: fcntl
  - Alarmer un handler correspondant à SIGURG: signal()
  - fcntl(sockt, F_SETOWN, getpid());
  - fcntl(nsockt, F_SETOWN, getpid());
  - signal(SIGURG, urg);

- Fonction service: lectures des chaines de caractères reçues depuis le client: lecture infinie
- Recevoir la taille du mot caché
- For(;;)
  - Lecture des chaines de caractères

  Il teste ensuite s'il a reçu le nombre de caractères urgents == taille du mot caché
- Écrire une fonction handler urg: recevoir le caractère urgent

- Dans le serveur:
- int cp=0;
- char mc[MAX];
- - void urg() …. …….
- Void service () ….
- Main ()
- {….
- ….
- sockt=socket(…
- ….
- nsockt=accept(…
  - fcntl(sockt, F_SETOWN, getpid());
  - fcntl(nsockt, F_SETOWN, getpid());
  - signal(SIGURG, urg);
- service()
- }

Création de Processus Concurrents en C

Notion de processus concurrents

- Répond aux questions :
  - Comment exécuter deux (ou plus) actions simultanément ?
    - Par exemple envoi et réception de données
  - Comment un serveur peut-il traiter les demandes de plusieurs clients simultanément?
- PID = Processus Identifier
  - Identifie un processus

103

Fork()

- Un des éléments les plus importants d'UNIX
- Permet de dupliquer un processus et ses données
  - Appelé par le processus père et lance un processus enfant
  - On obtient 2 processus **indépendants**
  - 1 appel du `fork` → 2 sorties (père + fils)
- Attention au partage de données !
  - Exemple : lecture/écriture des mêmes fichiers par plusieurs processus

104

Utilisation de fork()

- Inclure `<unistd.h>`
- `pid_t fork(void)`
  - Retourne `pid_t` :
    - `-1` en cas de problème
    - `0` dans le fils
    - Le PID du fils dans le père
- Après le fork :
  - Le père connaît le PID du fils
  - Le fils ne connaît pas le PID du père
    - Voir fonctions auxiliaires …

105

Exemple de fork()

```
#include <unistd.h>
…

/* Dans une fonction du père */
…
Int pid;
…
/* Création d'un fils */
if ((pid=fork())==0)
    { /* Code du fils */
      printf("Fils : Mon PID est %d\n", getpid());
    }
 else
    {   /* Vérification de la création */
        if (pid==-1)
            { /* Il y a eu un problème à la création */
              printf("PB fork\n"); exit(-1);
            }
        else
            { /* Code du père après la création du fils */
              printf("Père : Mon PID est %d, celui de mon fils est %d\n", getpid(),
pid);
            }
    }
// reste du code …
```

création d'un fils

code du fils

106

Fonctions auxiliaires

- `pid_t getpid(void)`
  - Retourne le PID du processus appelant
- `pid_t getppid(void)`
  - Retourne le PID du père du processus appelant
- `int wait(int *code_retour)`
  - Bloque un processus en attente de la terminaison d'un fils
  - Retourne le `pid` du fils terminé et son code de retour
  - Retourne `-1` si tous les fils sont déjà terminés
- `int waitpid(int pid, int *code_retour)`
  - Idem `wait` mais permet de choisir le `pid` du fils
- `int sleep(int t)`
  - Endort un processus pour `t` secondes
- `void exit(int status)`
  - Met fin à un processus et retourne le code status
  - Envoi le signal `SIGCHLD` au père

107

Pour éviter les problèmes …

- A la création d'un fils, le père et le fils possèdent pratiquement le même environnement :
  - Différences :
    - La valeur retournée par le fork
    - Leur PID
    - Les blocages de fichiers ne sont pas hérités
    - Les alarmes ne sont pas héritées
- Donc :
  - Un père et un fils possèdent TOUS LES DEUX les descripteurs des mêmes sockets
    - Pour éviter les situations d'interblocage, un des deux doit fermer les ressources qu'il n'utilise pas !
  - Exemple pour un serveur :
    - Si le processus père bloque sur la primitive `accept` en attente de clients et que les fils se chargent de la communication avec un client :
      - Pour le père :
        - Le `accept` retourne le descripteur de la socket de service
        - Création d'un fils
        - Fermeture de la socket de service
      - Pour le fils :
        - Fermeture de la socket d'écoute

108

- Tout processus unix lorsqu'il se termine devient zombie; n'occupe plus qu'un emplacement dans la table de process (plus de programme ni données; il ne réagit pas aux interruptions)
- => aussi longtemps que son père n'a pas pris connaissance de sa terminaison

109

- signal(SIGCLD, SIG_IGN);
- signal(SIGCHLD,sigchld); => ignorer:*
- signal(SIGCHLD, SIG_IGN)

110

- TCP/Lorsqu'on fait une lecture (recv): la primitive revient quand au moins un octet a été lu;- pas tous les octets ont été lus; il faut parfois boucler
- recv (recvfrom) retourne:
  - -1 si la connexion est rompue ou n'existe pas
  - >0 on a reçu un certain nombre d'octets
  - 0 l'application distante a fait un close
  - => utiliser la variable errno : indique l'erreur

111

Communication entre processus

- 4 moyens :
  - Les signaux
    - La forme la plus élémentaire de communication entre processus
    - Permet d'interrompre un processus
  - Les tubes
    - Permet l'échange d'informations entre processus
    - Ne permet pas d'interrompre un processus
  - Les fichiers
    - Attention aux accès concurrents !
      - Nécessite l'emploi de mécanismes d'exclusion mutuelle
  - Les sockets …
    - Et oui encore elles … et vous savez faire !

112

Les signaux

- Les signaux sont des interruptions logicielles
- Ils peuvent être envoyés par des processus ou le système
- Par défaut, un processus se termine à la réception d'un signal
- Il est possible de définir un comportement personnalisé à la réception d'un signal (excepté `SIGKILL`)
- Nécessite l'importation de `<signal.h>`

113

Les différents types de signaux

- Les signaux sont identifiés par un code :
  (liste non exhaustive)
  - `SIGINT`
    - Interruption (ex: Ctrl+C)
  - `SIGKILL`
    - Tue le processus
  - `SIGUSR1` et `SIGUSR2`
    - Signaux dont la fonction est libre
  - `SIGALRM`
    - Réception d'une alarme (ex: réveil)
  - `SIGCHLD`
    - Indique au père la mort d'un fils
  - `SIGPIPE`
    - Tentative invalide d'écriture dans un tube (ex: plus aucun lecteur)
  - `SIGTERM`
    - Demande au processus de se terminer
    - Attention : Ne pas confondre avec SIGKILL. SIGKILL ne demande rien …, il tue !
  - …

114

Envoi de signaux

- `int kill(int pid, int type_signal)`
  - Permet d'envoyer au processus désigné par `pid` le signal `type_signal`
- `long alarm(long duree)`
  - Demande au système d'envoyer au processus appelant le signal `SIGALRM` une fois la durée `duree` écoulée.

115

Réception de signaux

- `int pause (void)`
  - Bloque un processus jusqu'à ce qu'il reçoive un signal
- Sinon :
  - `int signal(int type_signal, void (handler *) action)`
    - Lance la fonction `action` dès que le signal `type_signal` est reçu
    - Il est possible de mettre `SIGIGN` pour l'action. Dans ce cas, le signal sera ignoré

116

Exemple d'emploi des signaux

```
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <stdio.h>

/* duree en secondes */
#define DUREE  5

/* Routine de traitement de SIGUSR1 */
void trait_SIGUSR1(int numero_signal)
{
	printf("J'ai recu SIGUSR1\n");
	/* repositionnement du signal */
	signal(SIGUSR1, trait_SIGUSR1);
}

/* Routine de traitement de SIGUSR2 */
void trait_SIGUSR2(int numero_signal)
{
	printf("J'ai recu SIGUSR2\n");
	/* repositionnement du signal */
	signal(SIGUSR2, trait_SIGUSR2);
}

/* Routine de traitement de SIGTERM */
void trait_SIGTERM(int numero_signal)
{
	printf("Je dois mourir !!!\n");
	exit(0);
}

/* Routine de traitement de SIGALRM */
void trait_SIGALRM(int numero_signal)
{
	printf("Ca sonne !!!\n");
	/* repositionnement du signal */
	signal(SIGALRM, trait_SIGALRM);
}

int main(int argc, char*argv[])
{
	/* Positionnement du comportement en réception de signaux */
	signal(SIGUSR1, trait_SIGUSR1);
	signal(SIGUSR2, trait_SIGUSR2);
	signal(SIGALRM, trait_SIGALRM);
	signal(SIGALRM, trait_SIGTERM);

	/* reste du main … */
	while (1)
	{
		printf("Je m'envoi SIGUSR1\n");
		kill(getpid(),SIGUSR1);
		printf("Je m'envoi SIGUSR2\n");
		kill(getpid(),SIGUSR2);
		printf("Je positionne l'alarme\n");
		alarm(DUREE);
	}
	printf("fin\n");
	exit(0);
}
```


Les tubes

- Permettent de créer un canal de communication entre deux processus
  - Les processus se partagent le tube
- Fonctionnement :
  - Création d'une paire de descripteurs de fichiers pointant sur l'entrée et la sortie du tube
  - Utilisation des fonctions `read` et `write` standards

Utilisation des tubes 1/2

- `int pipe(int tube[])`
  - Crée la paire de descripteurs dans le tableau `tube` :
    - `tube[0]` prend le descripteur de lecture
    - `tube[1]` prend le descripteur d'écriture
  - Retourne :
    - `-1` en cas de problème, `0` sinon
- Attention :
  - La lecture dans un tube est destructrice
    - L'information lue par un processus est retirée du tube, elle n'est donc plus disponible ensuite pour les autres processus !
    - Attention de créer le tube avant le `fork()`
      - Sinon le fils ne connait pas le tube …

119

Utilisation des tubes 2/2

- Du bon usage des tubes :
  - Si un processus n'a pas besoin de lire (resp. écrire), il est conseillé de lui faire fermer le descripteur de lecture (resp. écriture)
    - Cela peut éviter qu'un processus attende infiniment une lecture …
- Le signal `SIGPIPE`
  - Indique à un processus qui écrit dans un tube qu'il n'y a plus de lecteur de ce tube
  - Conseil :
    - La gestion de ce signal permet de détecter que le processus écrivain se termine …

120